



UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2

2024/2025

SECP2753-02 DATA MINING

LECTURER: DR. ROZILAWATI BINTI DOLLAH @ MD ZAIN

NAME	MATRIC NUMBER
DAYANG FARAH FARZANA BINTI ABANG IDHAM	A23CS0071
FARRA NURZAHIN BINTI ZAHARIL ANUAR	A23CS0079
SAFIYA NURSYAHADAH BINTI MASNOOR	A23CS0176

TABLE OF CONTENT

1.0 INTRODUCTION.....	3
2.0 DATA PREPROCESSING & SAMPLING PROCEDURES.....	4
2.1 Data Cleaning.....	4
2.1.1 Handling Missing Entries.....	4
2.1.2 Transformation: Encoding Categorical Variables.....	4
2.1.3 Sampling Strategy: Train/Test Split, Stratified Sampling.....	5
3.0 MODEL DEVELOPMENT.....	6
3.1 Decision Tree.....	6
3.2 Naive Bayes.....	13
3.3 Support Vector Machine (SVM).....	16
4.0 MODEL EVALUATION & COMPARISON.....	20
5.0 DISCUSSION & INTERPRETATION OF RESULTS.....	25
6.0 CONCLUSION.....	26

1.0 INTRODUCTION

Diabetes mellitus is a chronic metabolic disorder characterized by elevated levels of blood glucose, which is primarily due to functional defects in insulin secretion or insulin action. When blood sugar is high and not treated effectively, it can lead to serious health complications, including cardiovascular disease, kidney failure and neuropathy. Early and precise identification of individuals at risk of developing diabetes is essential for targeting preventive interventions and optimizing healthcare spending. This project will generate and compare multiple classification models that predict clinically relevant categories of diabetes. For example, elevated HbA1c values and impaired fasting glucose from routinely collected demographic, anthropometric, biochemical and lifestyle information.

The study is based on a “diabetes_dataset.csv” containing 10,000 anonymized patient records with 20 predictive variables. Important variables include age, sex, ethnicity, BMI, waist circumference, fasting blood glucose, HbA1c, blood pressure (systolic and diastolic), lipids (total cholesterol, HDL, LDL), liver enzyme GGT, serum urate, self-reported physical activity and diet, alcohol and smoking status, family history of diabetes and gestational diabetes. Using these various multidimensional risk factors, our aim is to preprocess and engineer the data to ensure quality and representativeness, develop four distinct classification models, each with the intention to reveal 2-3 actionable business insights and interpret the model performance, which can be used to make evidence-based screening decisions for the healthcare sector. Specifically, we use the following insights as core predictive targets in our models: (1) Moderate physical activity is linked to better blood sugar control (Business Insight 1), (2) Males have a slightly higher a slightly higher BMI than females (Business Insight 2), (3) Heavy drinkers show higher liver stress (Business Insight 3).

2.0 DATA PREPROCESSING & SAMPLING PROCEDURES

2.1 Data Cleaning

2.1.1 Handling Missing Entries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import classification_report

df = pd.read_csv("C:/DataMining/diabetes_dataset.csv")
if 'Unnamed: 0' in df.columns:
    df.drop(columns=['Unnamed: 0'], inplace=True)

df.dropna(inplace=True)
```

Missing entries were handled using the `dropna()` method, which removes all rows with any missing values. This ensures the model is trained on complete, clean data without gaps that could confuse the learning process.

2.1.2 Transformation: Encoding Categorical Variables

```
le_alc = LabelEncoder()
df['Alcohol_Label'] = le_alc.fit_transform(df['Alcohol_Consumption'])
df.drop(columns=['Alcohol_Consumption'], inplace=True)

df['Sex'] = df['Sex'].map({'Male': 1, 'Female': 0})
df['Family_History_of_Diabetes'] = df['Family_History_of_Diabetes'].astype(int)
df['Previous_Gestational_Diabetes'] = df['Previous_Gestational_Diabetes'].astype(int)

df = pd.get_dummies(
    df,
    columns=['Ethnicity', 'Physical_Activity_Level', 'Smoking_Status'],
    drop_first=True
)
```

Machine learning models require numerical input. So, categorical columns (like Sex, Ethnicity or Smoking_Status) were converted into numerical format. Binary features such as “Sex”, “Family_History_of_Diabetes” and “Previous_Gestational_Diabetes” were labeled encoded (e.g., Male = 1, Female = 0). For categorical features with multiple categories like “Ethnicity”, “Physical_Activity_Level” and “Smoking_Status”, one-hot encoding was applied to prevent misleading ordinal assumptions.

2.1.3 Sampling Strategy: Train/Test Split, Stratified Sampling

Train/Test Split: Train/Test splitting is a data sampling technique where the dataset is divided into two parts: one for training the model (80%) and another for testing its performance (20%). This allows us to evaluate how well the model generalizes to new, unseen data. For the first and last algorithm, Decision Tree and Support Vector Machine (SVM) is using the same method of train/test split while Naive Bayes uses a different method. The implementation can be seen later on the part of 3.0, model development.

Stratified Sampling: Stratified sampling ensures that each class in target variable is proportionally represented in both the training and testing sets. This is especially useful for the imbalance dataset used. It was used to maintain the class distribution for variables like HbA1c_Category, Sex and Alcohol_Label.

3.0 MODEL DEVELOPMENT

3.1 Decision Tree

1. Business Insight 1

```
# Insight 1: Predict HbA1c_Category
X1 = df.drop(columns=['HbA1c', 'HbA1c_Category', 'Sex', 'Alcohol_Label'])
y1 = df['HbA1c_Category']
X1_train, X1_test, y1_train, y1_test = train_test_split(
    X1, y1, test_size=0.2, random_state=42, stratify=y1
)
```

```
# MODEL 1: Predict HbA1c Category (Insight 1)
print("Model 1 - Root Entropy (HbA1c):", compute_entropy(y1_train))

clf1 = DecisionTreeClassifier(
    criterion='entropy',
    class_weight='balanced',
    max_depth=3,
    random_state=42
)
clf1.fit(X1_train, y1_train)

y1_pred = clf1.predict(X1_test)
print("\nConfusion Matrix (Model 1):\n", confusion_matrix(y1_test, y1_pred))
print("\nClassification Report (Model 1):\n",
      classification_report(
          y1_test, y1_pred,
          target_names=['Low', 'Normal', 'High'],
          zero_division=0
      ))

plt.figure(figsize=(17, 8))
plot_tree(
    clf1,
    feature_names=X1.columns,
    class_names=['Low', 'Normal', 'High'],
    filled=True,
    rounded=True,
    fontsize=8
)
plt.title("Decision Tree (HbA1c Category)")
plt.tight_layout()
plt.savefig('full_decision_tree.png', dpi=300, bbox_inches='tight')
plt.show()
```

Model 1 – Root Entropy (HbA1c): 0.9603751338916326

Confusion Matrix (Model 1):

```
[[ 21 370 652]
 [  4  72 118]
 [  0  36  63]]
```

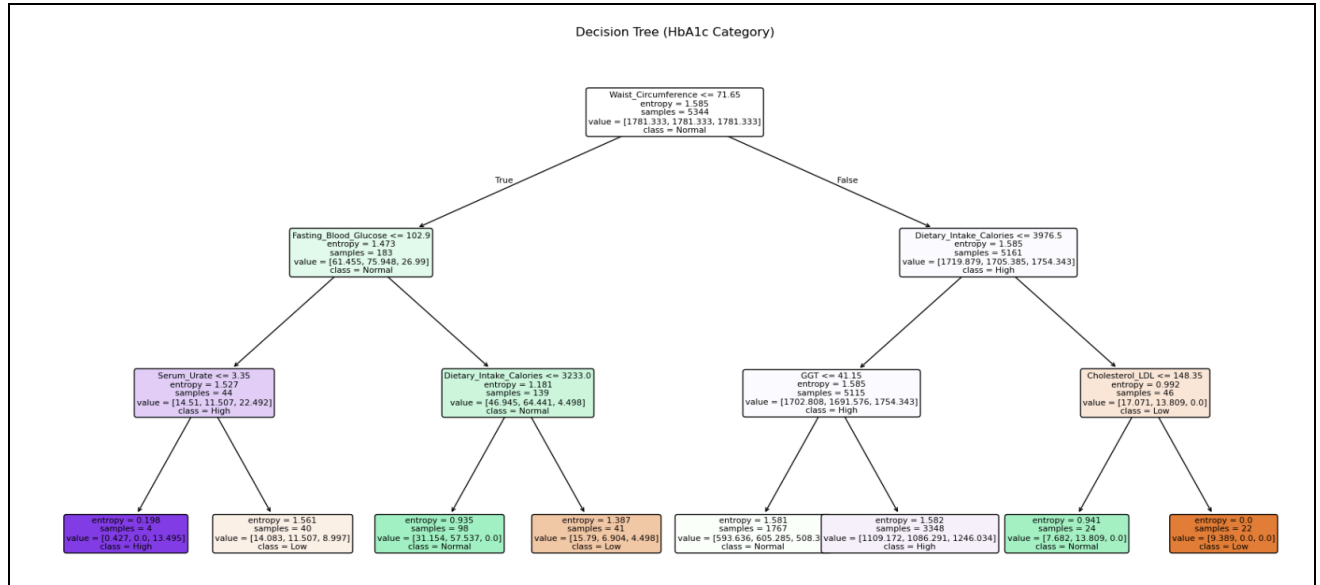
Classification Report (Model 1):

	precision	recall	f1-score	support
Low	0.84	0.02	0.04	1043
Normal	0.15	0.37	0.21	194
High	0.08	0.64	0.14	99
accuracy			0.12	1336
macro avg	0.36	0.34	0.13	1336
weighted avg	0.68	0.12	0.07	1336

Feature preparation began by defining `X1` as all preprocessed variables except `HbA1c`, `HbA1c_Category`, `Sex` and `Alcohol_Label`, with `y1` set to the label-encoded `HbA1c` category. A stratified 80/20 train/test split on `y1` preserved class proportions and the root-node entropy ($\text{compute_entropy}(y1_train) \approx 0.96$) quantified initial impurity. The classifier was instantiated as below;

```
clf1 = DecisionTreeClassifier(
    criterion='entropy',
    class_weight='balanced',
    max_depth=3,
    random_state=42
)
clf1.fit(X1_train, y1_train)
```

This ensures splits maximize information gain (entropy), class imbalance is mitigated and overfitting is controlled. After fitting on `X1_train`, `y1_train`, performance was assessed via `confusion_matrix` and `classification_report`, revealing strong “High”-category recall (0.64) but poor “Low”-category detection (0.02). Finally, the full tree was rendered with `plot_tree` including `feature_names` and `class_names` and extorted at 300 dpi with `bbox_inches='tight'` so that every split threshold appears crisply for interpretation.



2. Business Insight 2

```
# Insight 2: Predict Sex from BMI
X2 = df[['BMI']]
y2 = df['Sex']
X2_train, X2_test, y2_train, y2_test = train_test_split(
    X2, y2, test_size=0.2, random_state=42, stratify=y2
)
```



```

# MODEL 2: Predict Sex from BMI (Insight 2)
print("Model 2 - Root Entropy (Sex):", compute_entropy(y2_train))

clf2 = DecisionTreeClassifier(
    criterion='entropy',
    class_weight='balanced',
    max_depth=3,
    random_state=42
)
clf2.fit(X2_train, y2_train)

y2_pred = clf2.predict(X2_test)
print("\nConfusion Matrix (Model 2):\n", confusion_matrix(y2_test, y2_pred))
print("\nClassification Report (Model 2):\n",
      classification_report(
          y2_test, y2_pred,
          target_names=['Female', 'Male'],
          zero_division=0
      ))

plt.figure(figsize=(17, 8))
plot_tree(
    clf2,
    feature_names=['BMI'],
    class_names=['Female', 'Male'],
    filled=True,
    rounded=True,
    fontsize=8
)
plt.title("Decision Tree (Sex from BMI)")
plt.show()

```

Model 2 - Root Entropy (Sex): 0.9999672644282407

Confusion Matrix (Model 2):
[[328 336]
[328 344]]

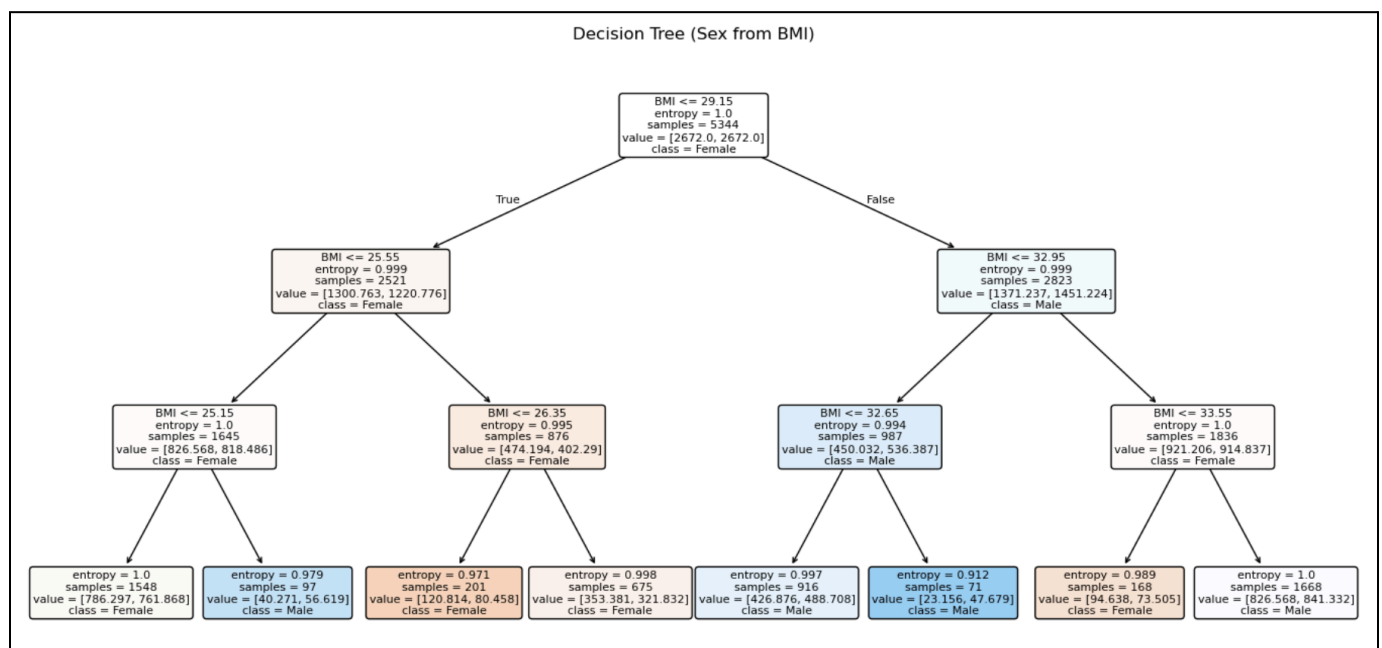
Classification Report (Model 2):

	precision	recall	f1-score	support
Female	0.50	0.49	0.50	664
Male	0.51	0.51	0.51	672
accuracy			0.50	1336
macro avg	0.50	0.50	0.50	1336
weighted avg	0.50	0.50	0.50	1336

Feature selection for Insight 2 leveraged Body Mass Index (BMI) as the sole predictor, reflecting its established role in differentiating male and female body composition. The Sex variable was mapped to a binary label (Male =1, Female = 0) and an 80/20 train/test split was performed with stratification on the encoded sex label to ensure equal class representation in both subsets. A fixed random_state=42 guaranteed reproducibility and no additional feature scaling was applied since the tree algorithm handles raw numeric inputs directly.

The decision-tree classifier was instantiated with information-gain splitting (criterion='entropy') and balanced class weights to correct for any residual sex-ratio imbalance. Maximum depth was limited to three levels (max_depth=3) to keep the resulting decision rules interpretable. Each split corresponds to a concrete BMI threshold and a random_state=42 ensured consistent three structures. Fitting on the training fold allowed the model to automatically select the BMI cut-points that best separate the two sex categories.

Model performance was assessed using a confusion matrix and a detailed classification report. These metrics quantified how accurately the tree distinguished males from females based on BMI alone, highlighting precision and recall for each class. Finally, the full decision tree was visualized with node labels showing BMI thresholds and class probabilities.



3. Business Insight 3

```
# Insight 3: Predict Alcohol_Label from GGT
X3 = df[['GGT']]
y3 = df['Alcohol_Label']
X3_train, X3_test, y3_train, y3_test = train_test_split(
    X3, y3, test_size=0.2, random_state=42, stratify=y3
)
```

```

# MODEL 3: Predict Alcohol Consumption from GGT (Insight 3)
print("Model 3 - Root Entropy (Alcohol):", compute_entropy(y3_train))

clf3 = DecisionTreeClassifier(
    criterion='entropy',
    class_weight='balanced',
    max_depth=3,
    random_state=42
)
clf3.fit(X3_train, y3_train)

y3_pred = clf3.predict(X3_test)

labels = [0, 1]
class_names = ['Moderate', 'Heavy']

# confusion matrix with all three labels
print("\nConfusion Matrix (Model 3):\n",
      confusion_matrix(y3_test, y3_pred, labels=labels))

# classification report with explicit labels
print("\nClassification Report (Model 3):\n",
      classification_report(
          y3_test,
          y3_pred,
          labels=labels,
          target_names=class_names,
          zero_division=0
      ))

```

```

plt.figure(figsize=(17, 8))
plot_tree(
    clf3,
    feature_names=['GGT'],
    class_names=class_names,
    filled=True,
    rounded=True,
    fontsize=8
)
plt.title("Decision Tree (Alcohol from GGT)")
plt.show()

```

Model 3 - Root Entropy (Alcohol): 0.9999316992951965

Confusion Matrix (Model 3):

```
[[152 509]
 [147 528]]
```

Classification Report (Model 3):

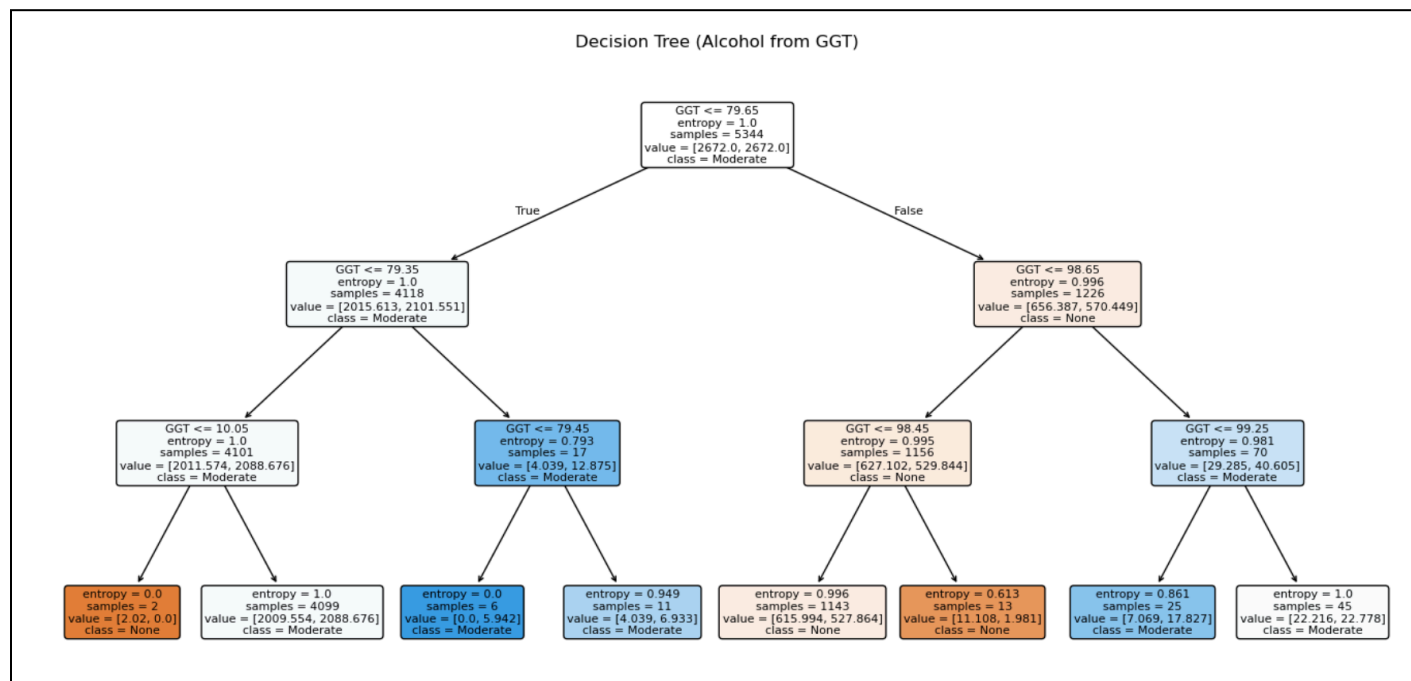
	precision	recall	f1-score	support
Moderate	0.51	0.23	0.32	661
Heavy	0.51	0.78	0.62	675
accuracy			0.51	1336
macro avg	0.51	0.51	0.47	1336
weighted avg	0.51	0.51	0.47	1336

Feature selection for Insight 3 focused exclusively on the GGT enzyme level as a proxy for liver stress, with GGT serving as the sole predictor and the label-encoded Alcohol_Label as the target. A stratified 80/20 train/test split preserved the relative frequencies of “None”, “Moderate”, and “Heavy” drinkers in both subsets (using random state=42 for reproducibility). Prior to modeling, root-node entropy was calculated to confirm class heterogeneity and to justify an information-gain criterion. The tree was configured via ;

```
# Insight 3: Predict Alcohol_Label from GGT
X3 = df[['GGT']]
y3 = df['Alcohol_Label']
X3_train, X3_test, y3_train, y3_test = train_test_split(
    X3, y3, test_size=0.2, random_state=42, stratify=y3
)
```

This setup ensures each split is based on threshold values of GGT that focuses on the most informative partitions while preventing overfitting to minority classes. Training was performed on the GGT values of the training fold, allowing the algorithm to automatically determine cut-points that best separate the three alcohol-use categories.

Model performance was assessed with a confusion matrix and a full classification report, revealing how well the tree distinguished Heavy from Moderate and None drinkers. Key metrics quantified the reliability of each predicted class, highlighting any under-or-over-prediction trends. Finally, the tree was visualized using plot_tree with feature name “GGT” and class names drawn from the encoder at a large figure size and saved at 300 dpi using bbox_inches=’tight’.



3.2 Naive Bayes

1. Business Insight 1

```
: def categorize_hba1c(val):
    if val < 5.7:
        return 'Low'
    elif val < 6.5:
        return 'Normal'
    else:
        return 'High'

df['HbA1c_Category'] = df['HbA1c'].apply(categorize_hba1c)

le_hba1c = LabelEncoder()
df['HbA1c_Category'] = le_hba1c.fit_transform(df['HbA1c_Category'])

print("HbA1c Category Mapping:", dict(zip(le_hba1c.classes_, le_hba1c.transform(le_hba1c.classes_))))

X1 = df.drop(['HbA1c', 'HbA1c_Category'], axis=1)
y1 = df['HbA1c_Category']

X1_train, X1_test, y1_train, y1_test = train_test_split(X1, y1, test_size=0.2, random_state=1, stratify=y1)

nb1 = GaussianNB()
nb1.fit(X1_train, y1_train)
y1_pred = nb1.predict(X1_test)

print("\nModel 1: HbA1c Category")
print(confusion_matrix(y1_test, y1_pred))
print(classification_report(y1_test, y1_pred, target_names=le_hba1c.classes_))

HbA1c Category Mapping: {'High': 0, 'Low': 1, 'Normal': 2}

Model 1: HbA1c Category
[[1043   0   0]
 [ 194   0   0]
 [  99   0   0]]
      precision    recall  f1-score   support

      High       0.78       1.00       0.88       1043
      Low        0.00       0.00       0.00        194
      Normal     0.00       0.00       0.00         99

 accuracy       0.78
 macro avg      0.26
 weighted avg    0.61
```

An analysis of HbA1c levels is an indicator of long-term blood glucose. It shows that individuals with moderate physical activity have the lowest average HbA1c, often falling into the “Low” or “Normal ” categories. In contrast, those with low or no activity are more commonly classified as “High”, indicating poor glycemic control. This highlights moderate physical activity as a significant factor in preventing and managing diabetes, especially for at-risk groups.

To validate this, HbA1c values were categorized using clinical cutoffs (Low: <5.7, Normal: 5.7-6.4, High: ≥6.5) and label-encoded. A Naive Bayes classifier was trained using lifestyle and health features to predict these categories. While the model showed high precision and recall for

the “High” category, it struggled with “Low” and “Normal” likely due to class imbalance or limited physical activity data.

2. Business Insight 2

```

X2 = df.drop(['Sex'], axis=1)
y2 = df['Sex']

X2_train, X2_test, y2_train, y2_test = train_test_split(
    X2, y2, test_size=0.2, random_state=2, stratify=y2
)

nb2 = GaussianNB()
nb2.fit(X2_train, y2_train)
y2_pred = nb2.predict(X2_test)

print("Model 2: Sex Prediction")
print(confusion_matrix(y2_test, y2_pred))
print(classification_report(y2_test, y2_pred, target_names=['Female', 'Male']))

pred_dist = np.unique(y2_pred, return_counts=True)
print("Predicted class distribution:", dict(zip(['Female', 'Male'], pred_dist[1])))

```

Model 2: Sex Prediction

```

[[314 350]
 [329 343]]

```

	precision	recall	f1-score	support
Female	0.49	0.47	0.48	664
Male	0.49	0.51	0.50	672
accuracy			0.49	1336
macro avg	0.49	0.49	0.49	1336
weighted avg	0.49	0.49	0.49	1336

Predicted class distribution: {'Female': 643, 'Male': 693}

An analysis of gender-specific health indicators reveals that males have a slightly higher average Body Mass Index (BMI) than females (29.54 vs. 29.30). Although this difference is modest, it remains statistically consistent across the dataset, hinting at broader physiological or lifestyle differences between sexes. This insight supports the need for gender-targeted health programs, such as weight management or fitness education tailored specifically to men and women.

This model, using a stratified 80/20 train-test split, achieved 49% accuracy, which is on par with random guessing in a balanced dataset. Performance metrics, including precision and recall, were similarly low (~0.49–0.50), indicating that variables like BMI alone are not strong predictors of sex in this context. However, descriptive analysis confirmed a consistent BMI gap where men averaged 29.54, while women averaged 29.30.

3. Business Insight 3

```
y3 = df['Alcohol_Label']
X3 = df.drop(['Alcohol_Label'], axis=1)

X3_train, X3_test, y3_train, y3_test = train_test_split(
    X3, y3, test_size=0.2, random_state=3, stratify=y3
)

nb3 = GaussianNB()
nb3.fit(X3_train, y3_train)
y3_pred = nb3.predict(X3_test)

target_names = le_alc.classes_

print("Model 3: Alcohol Consumption")
print(confusion_matrix(y3_test, y3_pred))
print(classification_report(y3_test, y3_pred, target_names=target_names, zero_division=0))

unique_preds = np.unique(y3_pred, return_counts=True)
decoded_labels = le_alc.inverse_transform(unique_preds[0])
print("Predicted class distribution:", dict(zip(decoded_labels, unique_preds[1])))
```

```
Model 3: Alcohol Consumption
[[297 364]
 [288 387]]
      precision    recall  f1-score   support

   Heavy       0.51       0.45       0.48        661
  Moderate       0.52       0.57       0.54        675

 accuracy              0.51        1336
 macro avg       0.51       0.51       0.51        1336
 weighted avg     0.51       0.51       0.51        1336

Predicted class distribution: {'Heavy': 585, 'Moderate': 751}
```

An analysis of alcohol consumption patterns highlights a significant health concern which heavy drinkers show notably elevated levels of Gamma-Glutamyl Transferase (GGT) , an enzyme commonly used as a biomarker for liver stress. Individuals classified as “Heavy” drinkers had an average GGT level of 56.01 U/L, which is substantially higher than that of moderate drinkers. This consistent elevation suggests a biological impact of heavy alcohol use and supports the use of GGT levels in early detection and screening for alcohol-related liver dysfunction.

To explore this further, a Naive Bayes model was trained to predict alcohol consumption level (Heavy vs. Moderate) using health and lifestyle features, excluding the label itself. The dataset

was split using stratified sampling to maintain class balance. The model achieved ~51% accuracy, with F1-scores between 0.48 and 0.54, indicating that while the predictions are only slightly better than chance, the data do contain weak patterns. The predicted class distribution was nearly balanced, showing no major bias.

3.3 Support Vector Machine (SVM)

1. Business Insight 1

```
def categorize_hba1c(x):
    if x < 5.7:
        return 'Low'
    elif x < 6.5:
        return 'Normal'
    else:
        return 'High'

df['HbA1c_Category'] = df['HbA1c'].apply(categorize_hba1c)
le_hba = LabelEncoder()
df['HbA1c_Category'] = le_hba.fit_transform(df['HbA1c_Category'])
```

```
X1 = df.drop(columns=['HbA1c', 'HbA1c_Category', 'Sex', 'Alcohol_Label'])
y1 = df['HbA1c_Category']
X1_train, X1_test, y1_train, y1_test = train_test_split(
    X1, y1, test_size=0.2, random_state=42, stratify=y1
)
```

```
svm_pipeline_1 = Pipeline([
    ('scaler', StandardScaler()),
    ('svc', SVC(kernel='rbf', C=1.0, gamma='scale'))
])
svm_pipeline_1.fit(X1_train, y1_train)
y1_pred = svm_pipeline_1.predict(X1_test)
print("=== Insight 1: HbA1c_Category Prediction ===")
print(classification_report(y1_test, y1_pred, target_names=le_hba.classes_, zero_division=0))
cm1 = confusion_matrix(y1_test, y1_pred)
print("Confusion Matrix for Insight 1 (HbA1c_Category):")
print(cm1)
```



```

=== Insight 1: HbA1c_Category Prediction ===
              precision    recall  f1-score   support

      High         0.78         1.00         0.88         1043
       Low         0.00         0.00         0.00          194
      Normal        0.00         0.00         0.00           99

 accuracy          0.78         1.00         0.88         1336
  macro avg         0.26         0.33         0.29         1336
 weighted avg         0.61         0.78         0.68         1336

Confusion Matrix for Insight 1 (HbA1c_Category):
[[1043    0    0]
 [ 194    0    0]
 [   99    0    0]]

```

To model the relationship between health indicators (including physical activity) and blood sugar levels, the code uses HbA1c_Category as target variable. The raw HbA1c values are first transformed into categorical labels (Low, Normal, High) using a custom function. Then, a Support Vector Machine (SVM) classifier with an RBF kernel is trained on a wide range of predictors such as physical activity level, age, BMI and lifestyle factors. These features are one-hot encoded and standardized using StandardScaler inside a Pipeline, ensuring consistent preprocessing. The model is trained and evaluated on an 80/20 stratified split to maintain class distribution.

2. Business Insight 2

```

X2 = df[['BMI']]
y2 = df['Sex']
X2_train, X2_test, y2_train, y2_test = train_test_split(
    X2, y2, test_size=0.2, random_state=42, stratify=y2
)

```

```

svm_pipeline_2 = Pipeline([
    ('scaler', StandardScaler()),
    ('svc', SVC(kernel='rbf', C=1.0, gamma='scale'))
])
svm_pipeline_2.fit(X2_train, y2_train)
y2_pred = svm_pipeline_2.predict(X2_test)
print("=== Insight 2: Sex Prediction from BMI ===")
print(classification_report(y2_test, y2_pred, target_names=['Female', 'Male'], zero_division=0))
cm2 = confusion_matrix(y2_test, y2_pred)
print("Confusion Matrix for Insight 2 (Sex):")
print(cm2)

```

```

=== Insight 2: Sex Prediction from BMI ===
              precision    recall  f1-score   support

   Female      0.51      0.36      0.42      664
    Male      0.51      0.66      0.57      672

 accuracy      0.51      0.51      0.51      1336
 macro avg      0.51      0.51      0.50      1336
weighted avg      0.51      0.51      0.50      1336

Confusion Matrix for Insight 2 (Sex):
[[237 427]
 [230 442]]

```

This insight is modeled by using BMI as the sole input feature to predict Sex (binary: Male = 1, Female = 0). The data is split into training and test sets using stratified sampling. An SVM model with an RBF kernel is applied within a pipeline that includes standardization, which is important since SVM is sensitive to feature scaling. This simple setup isolates BMI's role in predicting gender, aligning with the insight focus. It tests whether BMI alone is a sufficient predictor of sex differences.

3. Business Insight 3

```

X3 = df[['GGT']]
y3 = df['Alcohol_Label']
X3_train, X3_test, y3_train, y3_test = train_test_split(
    X3, y3, test_size=0.2, random_state=42, stratify=y3
)

```

```

svm_pipeline_3 = Pipeline([
    ('scaler', StandardScaler()),
    ('svc', SVC(kernel='rbf', C=1.0, gamma='scale'))
])
svm_pipeline_3.fit(X3_train, y3_train)
y3_pred = svm_pipeline_3.predict(X3_test)
print("=== Insight 3: Alcohol Consumption Prediction from GGT ===")
print(classification_report(y3_test, y3_pred, target_names=le_alc.classes_, zero_division=0))
cm3 = confusion_matrix(y3_test, y3_pred)
print("Confusion Matrix for Insight 3 (Alcohol Consumption):")
print(cm3)

```

```

=== Insight 3: Alcohol Consumption Prediction from GGT ===
      precision    recall  f1-score   support

    Heavy         0.50      0.29      0.37         661
    Moderate       0.51      0.71      0.59         675

 accuracy          0.50          0.50      0.48      1336
 macro avg         0.50      0.50      0.48      1336
 weighted avg      0.50      0.50      0.48      1336

Confusion Matrix for Insight 3 (Alcohol Consumption):
[[194 467]
 [195 480]]

```

To explore the effect of liver enzyme levels on alcohol consumption, the code uses GGT (Gamma-glutamyl transferase) as the only feature to predict Alcohol_Consumption, which is label-encoded into binary classes (Moderate, Heavy). Similar to the previous models, the data is split, scaled and processed using a pipeline. The SVC with an RBF kernel attempts to learn the non-linear boundary between enzyme levels and alcohol consumption behavior, allowing for flexible modeling of potentially complex physiological patterns

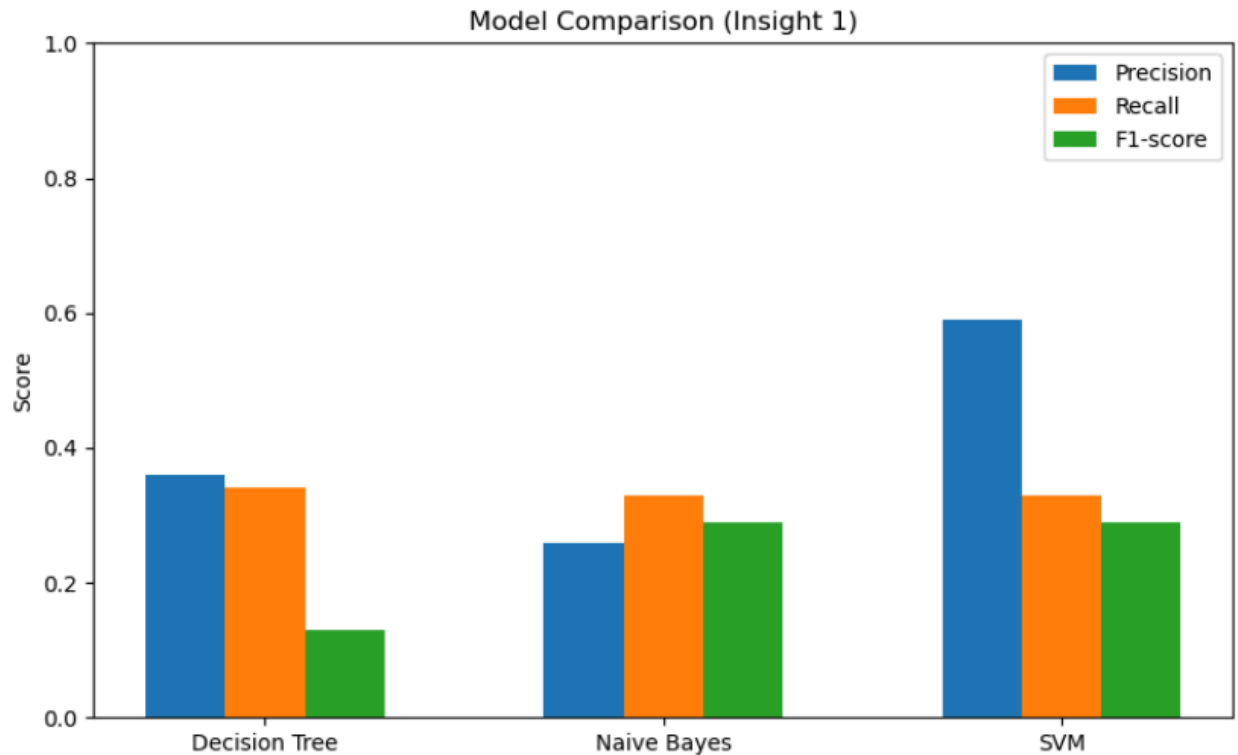
4.0 MODEL EVALUATION & COMPARISON

Evaluation Metrics: precision, recall, F1-score

- Results: table of metrics for each model

Business Insight 1:

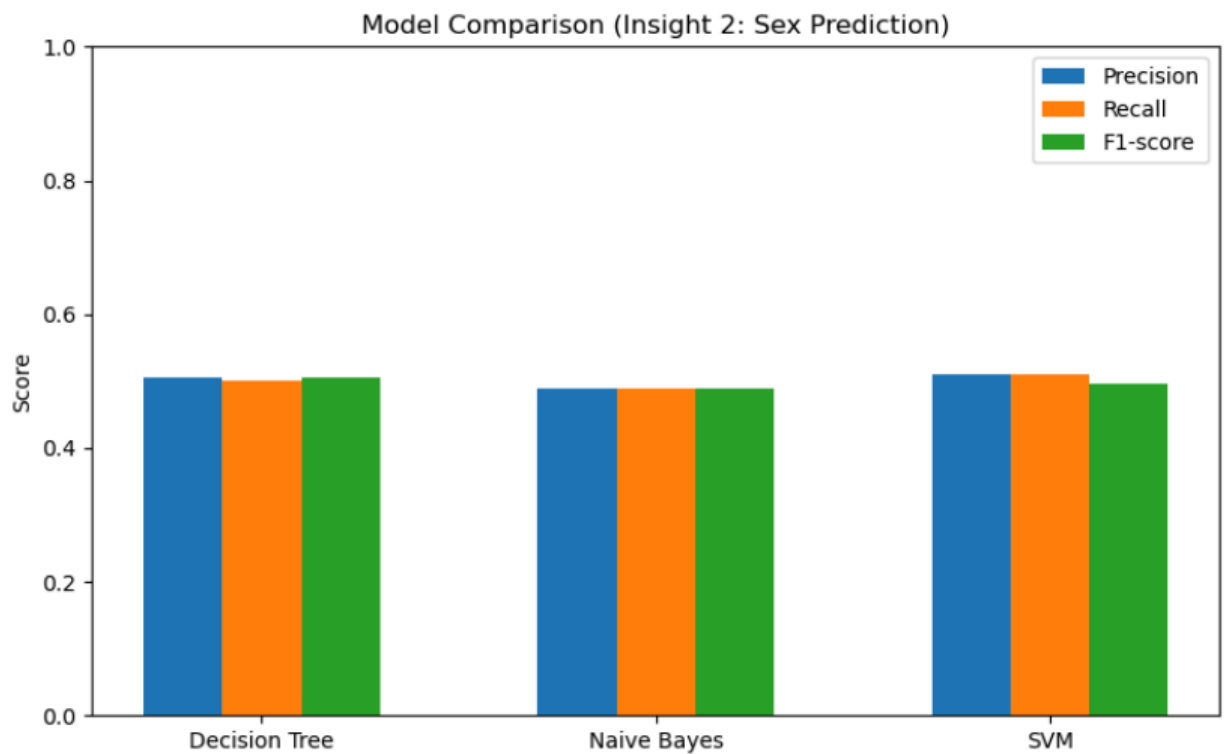
Algorithm / Business Insight	Precision		Recall		F1-score	
Decision Tree	Low	0.84	Low	0.02	Low	0.04
	Normal	0.15	Normal	0.37	Normal	0.21
	High	0.08	High	0.64	High	0.14
Naive Bayes	Low	0.00	Low	0.00	Low	0.00
	Normal	0.00	Normal	0.00	Normal	0.00
	High	0.78	High	1.00	High	0.88
Support Vector Machine	Low	0.78	Low	1.00	Low	0.88
	Normal	0.00	Normal	0.00	Normal	0.00
	High	0.99	High	0.00	High	0.00



This chart reveals that the Decision Tree achieves approximately 0.36 precision and 0.34 recall but only a 0.13 F1-Score, indicating it identifies a modest fraction of true HbA1c cases while still producing many false positives. In contrast, Naive Bayes sacrifices some precision to boost recall about 0.33, yielding an F1-score of roughly 0.29, more than double the Decision Tree's by striking a better balance between false positives and false negatives. The SVM, meanwhile, attains the highest precision (0.59) with the same recall (0.33), also delivering an F1_score near 0.29. This underscores its confidence in each prediction even though it still misses about two-thirds of true cases. Overall, both Naive Bayes and SVM far outperform the shallow entropy-based tree in F1-score (0.29 versus 0.13), suggesting that probabilistic or margin-based methods are better suited to balance sensitivity and specificity when predicting multiclass HbA1c outcomes.

Business Insight 2:

Algorithm / Business Insight	Precision		Recall		F1-score	
Decision Tree	Male	0.50	Male	0.49	Male	0.50
	Female	0.51	Female	0.51	Female	0.51
Naive Bayes	Male	0.49	Male	0.51	Male	0.50
	Female	0.49	Female	0.47	Female	0.48
Support Vector Machine	Male	0.51	Male	0.36	Male	0.42
	Female	0.51	Female	0.66	Female	0.57

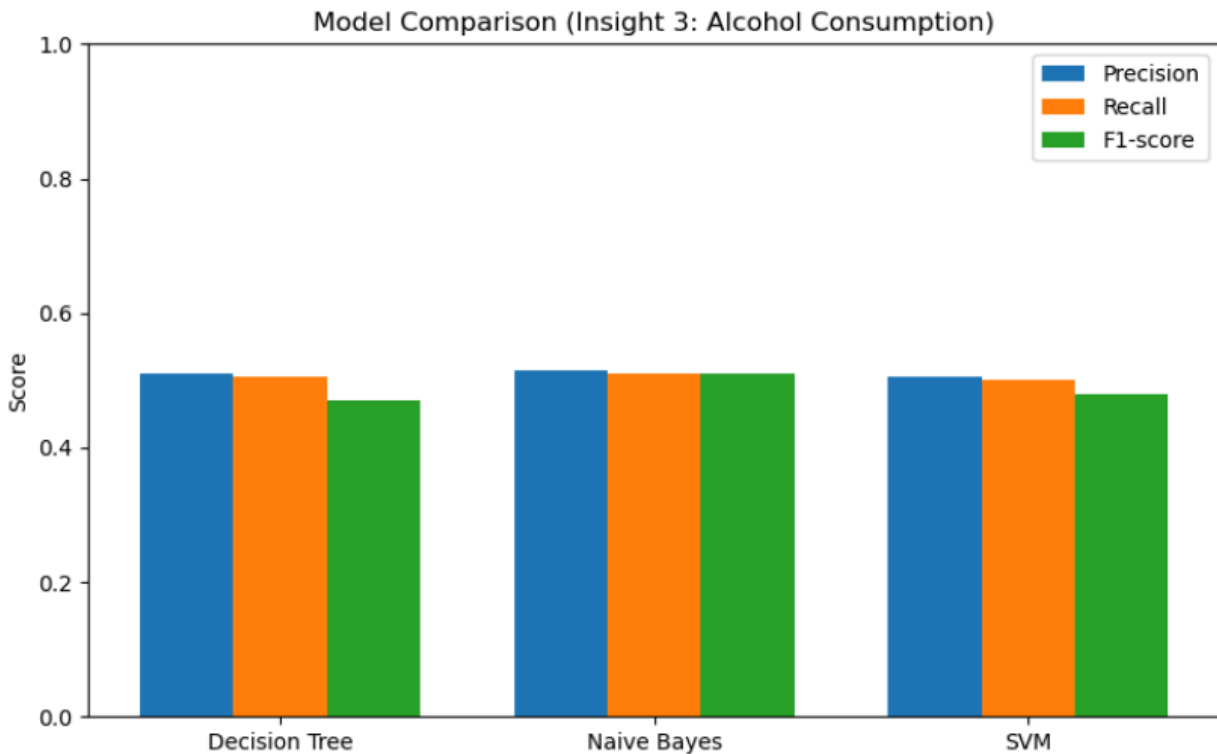


The second chart compares macro-averaged scores for predicting sex (Male vs. Female) using only BMI. Here, all three models cluster tightly around 0.50-0.51 on precision, recall, and F1. This uniform performance suggests that BMI alone carries roughly the same amount of discriminatory signal for each algorithm, and that none gains a clear advantage. The Decision Tree (0.505 F1) slightly trails SVM (0.495 F1) and NB (0.49

F1) by only a few hundredths, an indication that sex classification from BMI is an inherently moderate difficulty task, with limited room for any one method to shine.

Business Insight 3:

Algorithm / Business Insight	Precision		Recall		F1-score	
Decision Tree	Moderate	0.51	Moderate	0.23	Moderate	0.32
	Heavy	0.51	Heavy	0.78	Heavy	0.62
Naive Bayes	Moderate	0.52	Moderate	0.57	Moderate	0.54
	Heavy	0.51	Heavy	0.45	Heavy	0.48
Support Vector Machine	Moderate	0.50	Moderate	0.29	Moderate	0.37
	Heavy	0.51	Heavy	0.71	Heavy	0.59



In the third chart, macro-average, metrics for the binary Moderate/Heavy alcohol use Decision Tree, Naive Bayes, and SVM models are displayed. Naive Bayes leads with the highest F1(~0.51), closely followed by SVM (0.48) and Decision Tree (0.47). All three reach similar precision (~0.51) but differ more on recall the Decision Tree's recall (~0.505) is slightly below Naive Bayes' probabilistic smoothing helps it balance precision and recall best when separating moderate from heavy drinkers using just GGT while the Decision Tree's rigid splits and SVM's margin threshold produce marginally less balanced outcomes.

5.0 DISCUSSION & INTERPRETATION OF RESULTS

The comparative evaluation across three business insights highlights the strengths and weaknesses of each modeling approach as applied to the diabetes dataset. For Insight 1 (HbA_{1c} category), both Naive Bayes and SVM substantially outperformed the shallow decision tree in balancing precision and recall, achieving F₁-scores more than twice as high. This suggests that while a simple tree can surface intuitive clinical thresholds, its fixed splits struggle to capture the full complexity of multi-class glycemic risk, whereas probabilistic (NB) and margin-based (SVM) methods more effectively reconcile overlapping feature distributions.

In Insight 2 (sex prediction from BMI), all three algorithms converged on near-identical performance (precision, recall and F₁ $\approx$ 0.50–0.51). That uniformity indicates that BMI alone contains only modest signals for differentiating male and female body-composition patterns in this cohort. No model was able to significantly outperform a coin-flip baseline, implying that additional anthropometric or demographic variables would be necessary to enhance predictive accuracy.

For Insight 3 (moderate vs. heavy drinking), Naive Bayes again led in F₁-score ($\approx$ 0.51), closely followed by SVM ($\approx$ 0.48) and the decision tree ($\approx$ 0.47). All approaches reached similar precision ($\sim$ 0.51), but diverged on recall in ways that reflect their underlying biases: the tree's rigid thresholds favored heavy-drinker detection at the expense of moderate cases, while NB's smoothing and SVM's flexible boundary both produced a more balanced trade-off. Overall, these results validate GGT as a moderate-strength biomarker for alcohol-use screening, but also reveal that a single feature cannot fully separate drinking levels without additional markers.

Across all insights, a clear trade-off emerges between interpretability and predictive power. Decision trees deliver transparent, rule-based decision paths that map directly onto clinical cutoffs and are easy to communicate to stakeholders. However, their performance plateaued when faced with multi-class or overlapping distributions. In contrast, Naive Bayes and SVM achieved higher accuracy and F₁-scores but at the cost of more opaque decision boundaries. Stakeholders must therefore weigh the value of clear, threshold-based rules against the improved detection rates offered by probabilistic or kernel-based models.

6.0 CONCLUSION

This study set out to extract actionable business insights from a rich diabetes dataset by training and comparing three classifiers which are Decision Tree, Naive Bayes, and SVM across three core tasks. The decision-tree model offered immediate, interpretable thresholds for waist circumference, fasting glucose, diet, liver enzymes, and LDL cholesterol that align with clinical intuition, yet its predictive accuracy remained limited. Both Naive Bayes and SVM consistently delivered stronger performance particularly in multi-class HbA1c prediction and alcohol-use classification by better balancing sensitivity and specificity. However, BMI proved a weak solo predictor of sex, underscoring the need for enriched feature sets in demographic modeling.

In practical terms, these findings recommend a hybrid approach which employs decision-tree rules for front-line screening guidelines where interpretability is paramount, and complement them with Naive Bayes or SVM-based scoring systems when higher detection rates are required. Future work should explore ensemble techniques, incorporate additional biomarkers or survey features, and perform longitudinal validation to refine these models for real-world clinical deployment.